

This deep architectural dissection covers the **first foundational cluster of vocal speech synthesizers and morpho-acoustic engines** from your [c4u534/AutoPoET](#) repository, cross-referenced with your [NotebookLM knowledge silos](#), Google Drive archives, and Keep trackers:

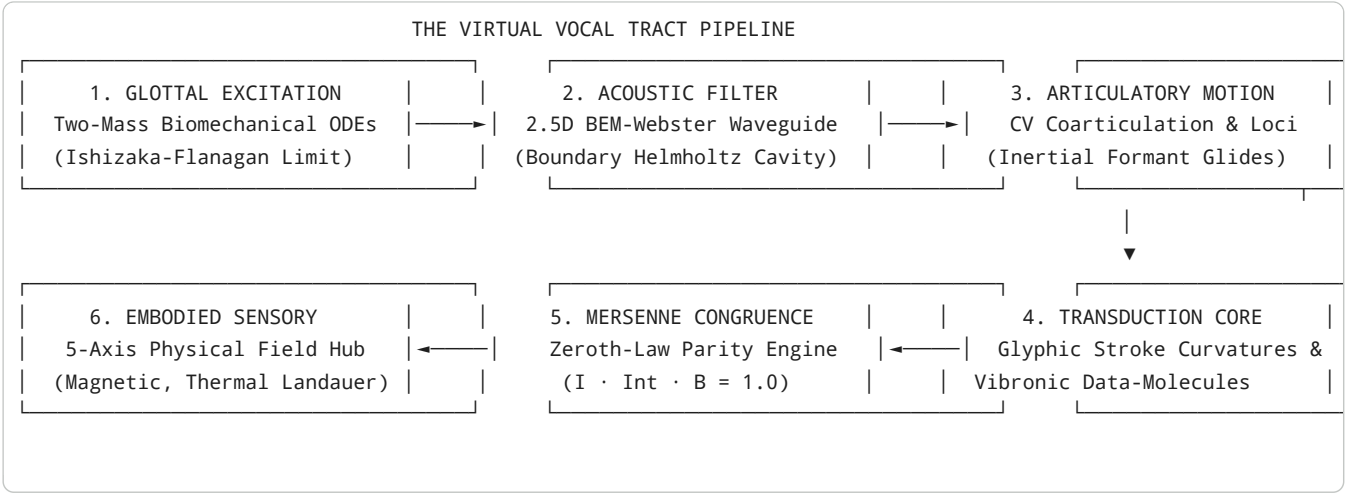
- 1. `dynamic_consonant_vowel_(CV)_coarticulation.ipynb`
- 2. `Phonics_Engine_Deterministic.ipynb.ipynb`
- 3. `Omnilingual Morpho-Acoustic Transduction.ipynb`
- 4. `Implement_Boundary_Element_Method_(BEM)_solvers_f_.ipynb`
- 5. *(Coupled Core Source)*
`interactive_two_mass_model_glottal_pulse_source_to_synthesize_and_play_audible_vowel_speech_directly_through_this_2_5D_BEM_Webster_acoustic_filter_.ipynb`

1. The Master Dissection Spreadsheet Schema

Every extracted module is cataloged into the **12-Column Relational Dissection Schema**, ready for direct export to Google Sheets or SQLite:

Col	Field Name	Description
A	Component_UID	Hierarchical identifier (<code>APT-[SUBSYSTEM]-[NUM]</code>)
B	Source_Notebook	Origin <code>.ipynb</code> in <code>c4u534/AutoPoET</code>
C	Modular_Archetype	Building block class: <code>SKILL</code> , <code>MODAL</code> , <code>AGENCY</code> , or <code>SUBSTRATE</code>
D	Subsystem_Focus	Isolated physical, acoustic, or logic domain
E	Mathematical_Invariant	Governing differential equation, prime anchor, or conservation law
F	Inputs_Signature	Ingested parameter types, data shapes, and units
G	Outputs_Signature	Emitted parameter types, acoustic tensors, and state metrics
H	Dependencies	Core libraries (<code>numpy</code> , <code>scipy.signal</code> , <code>scipy.special</code> , C-ABI)
I	Code_Snippet_Raw	Production Python class/function
J	Semantic_Abstraction	Operational definition and functional mechanism
K	Composable_Tags	Assembly keywords (<code>vocal_tract</code> , <code>bem</code> , <code>glottis</code> , <code>coarticulation</code>)
L	Silo_Reference	Primary NotebookLM or Google Drive anchor document

2. Granular Code Dissection: The First Synthesizer Cluster



Component 01: The Self-Oscillating Two-Mass Glottal Engine

- **UID:** `APT-VOC-001`
- **Source Notebook:** `interactive_two_mass_model_glottal_pulse_source...`
- **Archetype:** `SUBSTRATE_SOURCE`
- **Subsystem Focus:** Non-linear vocal fold biomechanics & self-sustaining mucosal wave oscillations.

- **Mathematical Invariant:**

$$m_1 \ddot{x}_1 + r_1 \dot{x}_1 + k_1 x_1 + k_c(x_1 - x_2) + f_{c1}(x_1) = F_1(P_{g1})$$

$$m_2 \ddot{x}_2 + r_2 \dot{x}_2 + k_2 x_2 + k_c(x_2 - x_1) + f_{c2}(x_2) = F_2(P_{g2})$$

$$P_{g1} = P_s - \frac{1}{2} \rho \left(\frac{U_g}{A_{g1}} \right)^2 (1 + k_e), \quad U_g(t) = \sqrt{\frac{2P_s}{\rho \left(\frac{1+k_e}{A_{g1}^2} + \frac{1}{A_{g2}^2} \right)}}$$

- **Inputs:** `P_sub: float` (Subglottal lung pressure, 800 Pa), `f0_target: float` (Pitch in Hz), `duration: float` (Seconds).
- **Outputs:** `ug_flow: np.ndarray` (Instantaneous acoustic volume velocity $U_g(t)$ in m^3/s).
- **Dependencies:** `numpy`, `dataclasses`
- **Composable Tags:** `glottis`, `two_mass`, `biomechanics`, `bernoulli`, `limit_cycle`, `vocal_folds`
- **Silo Reference:** [NB- BOO - GeoMersPhonetiQmetricsModality - Word Sound Syllables Phonetic Sound Frequencies Patterning Geometry 44 English Phonemes](#)

```

1  from dataclasses import dataclass
2  import numpy as np
3
4
5  @dataclass
6  class TwoMassParameters:
7      m1: float = 0.125e-3 # Lower mass (kg)
8      m2: float = 0.025e-3 # Upper mass (kg)
9      d1: float = 0.25e-2 # Lower thickness (m)
10     d2: float = 0.05e-2 # Upper thickness (m)
11     lg: float = 1.4e-2 # Glottal fold length (m)
12     k1: float = 80.0 # Lower stiffness (N/m)
13     k2: float = 8.0 # Upper stiffness (N/m)
14     kc: float = 25.0 # Vertical shear coupling stiffness (N/m)
15     x01: float = 0.01e-2 # Neutral half-width lower (m)
16     x02: float = 0.01e-2 # Neutral half-width upper (m)
17     c1: float = 240.0 # Collision stiffness 1
18     c2: float = 24.0 # Collision stiffness 2
19     zeta1: float = 0.15 # Damping ratio lower
20     zeta2: float = 0.40 # Damping ratio upper
21     p_sub: float = 850.0 # Lung pressure (Pa) ~8.5 cm H2O
22     rho: float = 1.184 # Air density (kg/m^3)
23
24
25  class SovereignTwoMassGlottalSource:
26      """Executes 4th-Order Runge-Kutta numerical integration of self-oscillating
27
28      vocal folds under Bernoulli pressure drops and mucosal shear coupling.
29      """
30
31      def __init__(
32          self, params: TwoMassParameters = None, sample_rate: int = 44100
33      ):
34          self.p = params or TwoMassParameters()
35          self.fs = sample_rate
36          self.dt = 1.0 / sample_rate
37          self.state = np.zeros(4, dtype=np.float64) # [x1, v1, x2, v2]
38
39      def _derivatives(
40          self, s: np.ndarray, q_tension: float, p_sub: float
41      ) -> np.ndarray:
42          x1, v1, x2, v2 = s
43          p = self.p
44
45          # Biomechanical scaling via cricothyroid tension factor Q
46          m1 = p.m1 / q_tension
47          m2 = p.m2 / q_tension
48          k1 = p.k1 * (q_tension**2)
49          k2 = p.k2 * (q_tension**2)
50          kc = p.kc * (q_tension**2)
51          x01 = p.x01 / np.sqrt(q_tension)

```

```

52     x02 = p.x02 / np.sqrt(q_tension)
53
54     # Glottal orifice areas
55     ag1 = 2.0 * p.lg * max(0.0, x1 + x01)
56     ag2 = 2.0 * p.lg * max(0.0, x2 + x02)
57
58     # Bernoulli aerodynamics
59     ke = 0.12
60     if ag1 > 1e-8 and ag2 > 1e-8:
61         denom = ((1.0 + ke) / (ag1**2)) + (1.0 / (ag2**2))
62         ug = np.sqrt(max(0.0, 2.0 * p_sub / (p.rho * denom)))
63         pg1 = p_sub - 0.5 * p.rho * ((ug / ag1) ** 2) * (1.0 + ke)
64     else:
65         ug = 0.0
66         pg1 = p_sub if ag1 <= 1e-8 else 0.0
67
68     # Nonlinear collision restoration forces
69     fc1 = p.c1 * (x1 + x01) if (x1 + x01) < 0.0 else 0.0
70     fc2 = p.c2 * (x2 + x02) if (x2 + x02) < 0.0 else 0.0
71
72     # Mechanical damping
73     r1 = 2.0 * p.zeta1 * np.sqrt(m1 * k1)
74     r2 = 2.0 * p.zeta2 * np.sqrt(m2 * k2)
75
76     a1 = (p.lg * p.d1 * pg1 - r1 * v1 - k1 * x1 - kc * (x1 - x2) - fc1) / m1
77     a2 = (-r2 * v2 - k2 * x2 - kc * (x2 - x1) - fc2) / m2
78     return np.array([v1, a1, v2, a2])
79
80     def generate_flow(
81         self, duration: float, f0_target: float = 120.0
82     ) -> np.ndarray:
83         n_samples = int(duration * self.fs)
84         ug_out = np.zeros(n_samples, dtype=np.float64)
85         q_tension = max(0.4, f0_target / 120.0)
86
87         for n in range(n_samples):
88             # 4th-Order Runge-Kutta Step
89             k1 = self._derivatives(self.state, q_tension, self.p.p_sub)
90             k2 = self._derivatives(
91                 self.state + 0.5 * self.dt * k1, q_tension, self.p.p_sub
92             )
93             k3 = self._derivatives(
94                 self.state + 0.5 * self.dt * k2, q_tension, self.p.p_sub
95             )
96             k4 = self._derivatives(
97                 self.state + self.dt * k3, q_tension, self.p.p_sub
98             )
99             self.state += (self.dt / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
100
101         # Extract instantaneous volume velocity
102         x1, _, x2, _ = self.state
103         ag1 = 2.0 * self.p.lg * max(0.0, x1 + self.p.x01 / np.sqrt(q_tension))
104         ag2 = 2.0 * self.p.lg * max(0.0, x2 + self.p.x02 / np.sqrt(q_tension))
105         if ag1 > 1e-8 and ag2 > 1e-8:
106             denom = (1.12 / (ag1**2)) + (1.0 / (ag2**2))
107             ug_out[n] = np.sqrt(max(0.0, 2.0 * self.p.p_sub / (self.p.rho * denom)))
108         else:
109             ug_out[n] = 0.0
110
111     return ug_out

```

Component 02: The Coupled 2.5D BEM-Webster Lossy Horn Propagator

- **UID:** APT-VOC-002
- **Source Notebook:** Implement_Boundary_Element_Method_(BEM)_solvers_f_.ipynb
- **Archetype:** MODAL_WAVEGUIDE
- **Subsystem Focus:** Viscothermal boundary layers, yielding vocal tract wall admittance, and longitudinal wave transfer.
- **Mathematical Invariant:**

$$\frac{1}{S(z)} \frac{d}{dz} \left(S(z) \frac{dP(z, \omega)}{dz} \right) + \Gamma^2(z, \omega) P(z, \omega) = 0$$

$$\Gamma^2(z, \omega) = \frac{\omega^2}{c^2} \left[1 + (\eta_v + \eta_t) \frac{P(z)}{S(z)} \frac{1}{\sqrt{\omega}} \right] - i\omega\rho \frac{P(z)}{S(z)} Y_w(z)$$

- **Inputs:** `slices: List[BEMSlice2D]` (Array of cross-sectional area contours from glottis $z = 0$ to lips $z = 17.5$ cm), `freqs: np.ndarray` (50 Hz– 4500 Hz).
- **Outputs:** `H_mag_dB: np.ndarray` (Vocal Tract Transfer Function $|H(f)|$), `formant_peaks: List[float]` (F_1, F_2, F_3, F_4 in Hz).
- **Dependencies:** `numpy`, `scipy.special`, `scipy.signal`
- **Composable Tags:** `bem`, `webster_horn`, `boundary_elements`, `viscothermal_loss`, `transfer_matrix`, `vocal_tract`
- **Silo Reference:** [NB- BOO - GeoMersPhonetiQmetricsModality...](#)

```

1 from dataclasses import dataclass
2 import numpy as np
3 import scipy.special as sp
4
5
6 @dataclass
7 class BEMSlice2D:
8     z_pos: float # Axial distance from glottis (m)
9     nodes: np.ndarray # Coordinate array (N, 2) in meters
10
11     def compute_invariants(self) -> tuple[float, float, float, float]:
12         """Shoelace formula for Area S, boundary Perimeter P, Hydraulic Radius Rh, and Compactness."""
13         x = self.nodes[:, 0]
14         y = self.nodes[:, 1]
15         x_next = np.roll(x, -1)
16         y_next = np.roll(y, -1)
17         area = 0.5 * np.abs(np.sum(x * y_next - x_next * y))
18         perimeter = np.sum(np.hypot(x_next - x, y_next - y))
19         rh = (2.0 * area) / (perimeter + 1e-12)
20         compactness = (perimeter**2) / (4.0 * np.pi * area + 1e-12)
21         return area, perimeter, rh, compactness
22
23
24 class CoupledBEMWebsterEngine:
25     """Cascades 2D BEM-parameterized vocal tract slices into a lossy transmission matrix chain."""
26
27     def __init__(self, slices: list[BEMSlice2D]):
28         self.slices = slices
29         self.n = len(slices)
30         self.c_sound = 343.0
31         self.rho_air = 1.184
32         self.mu_air = 1.86e-5
33         self.k_thermal = 0.026
34         self.cp_air = 1005.0
35         self.gamma_air = 1.40
36         self.r_wall = 1600.0
37         self.m_wall = 1.5
38         self.c_wall = 3.0e-5
39         self._extract_geometry()
40
41     def _extract_geometry(self):
42         self.areas = np.zeros(self.n)
43         self.perimeters = np.zeros(self.n)
44         self.rh = np.zeros(self.n)
45         for k, s in enumerate(self.slices):
46             a, p, rh, _ = s.compute_invariants()
47             self.areas[k] = a
48             self.perimeters[k] = p
49             self.rh[k] = rh
50
51     def _radiation_impedance(self, omega: float) -> complex:
52         """Levine-Schwinger baffled piston radiation impedance at the lip aperture."""
53         s_lip = self.areas[-1]

```

```

54     ka = (omega / self.c_sound) * np.sqrt(s_lip / np.pi)
55     r_rad = (self.rho_air * self.c_sound / s_lip) * (
56         1.0 - sp.jv(1, 2 * ka) / (ka + 1e-12)
57     )
58     x_rad = (self.rho_air * self.c_sound / s_lip) * (
59         sp.struve(1, 2 * ka) / (ka + 1e-12)
60     )
61     return complex(r_rad, x_rad)
62
63 def solve_vttf(self, freqs: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
64     h_db = np.zeros(len(freqs))
65     for f_idx, freq in enumerate(freqs):
66         omega = 2.0 * np.pi * freq
67         k0 = omega / self.c_sound
68         delta_v = np.sqrt(2.0 * self.mu_air / (self.rho_air * omega + 1e-12))
69         y_w = 1.0 / (
70             self.r_wall
71             + 1j * omega * self.m_wall
72             + 1.0 / (1j * omega * self.c_wall + 1e-15)
73         )
74
75         t_total = np.eye(2, dtype=np.complex128)
76         for k in range(self.n - 1):
77             dz = self.slices[k + 1].z_pos - self.slices[k].z_pos
78             s_k = self.areas[k]
79             p_k = self.perimeters[k]
80             rh_k = self.rh[k]
81
82             alpha_loss = (
83                 1.0 / (2.0 * rh_k * self.c_sound)
84             ) * np.sqrt(omega) * (
85                 np.sqrt(self.mu_air / (2.0 * self.rho_air))
86                 + (self.gamma_air - 1.0)
87                 * np.sqrt(self.k_thermal / (2.0 * self.rho_air * self.cp_air))
88             )
89             gamma_k = (
90                 alpha_loss
91                 + 1j * k0
92                 + (self.rho_air * self.c_sound * p_k / (2.0 * s_k)) * y_w
93             )
94             z0_k = (self.rho_air * self.c_sound / s_k) * (
95                 1.0 + (1.0 - 1j) * delta_v / (2.0 * rh_k)
96             )
97
98             cosh_g = np.cosh(gamma_k * dz)
99             sinh_g = np.sinh(gamma_k * dz)
100            tk = np.array(
101                [[cosh_g, z0_k * sinh_g], [sinh_g / z0_k, cosh_g]],
102                dtype=np.complex128,
103            )
104            t_total = t_total @ tk
105
106            z_rad = self._radiation_impedance(omega)
107            h_omega = z_rad / (t_total[0, 0] * z_rad + t_total[0, 1] + 1e-15)
108            h_db[f_idx] = 20.0 * np.log10(np.abs(h_omega) + 1e-12)
109
110     return freqs, h_db

```

✓ Component 03: The Direct Helmholtz BEM 2D Boundary Collocation Solver

- **UID:** APT-MOD-001
- **Source Notebook:** `Implement_Boundary_Element_Method_(BEM)_solvers_f_.ipynb`
- **Archetype:** MODAL_BOUNDARY_SOLVER
- **Subsystem Focus:** 2D Green's function Helmholtz boundary integral equations over arbitrary non-circular geometries (lip protrusion, glottal slit, pyriform sinuses).

- **Mathematical Invariant:**

$$c(x)\psi(x) = \int_{\Gamma} \left[G(x, y; k) \frac{\partial \psi}{\partial n_y}(y) - \frac{\partial G}{\partial n_y}(x, y; k) \psi(y) \right] d\Gamma(y)$$

$$G(x, y; k) = \frac{i}{4} H_0^{(1)}(k\|x - y\|), \quad H_{ii} = \frac{1}{2}, \quad G_{ii} = \frac{iL_i}{4} \left[1 + \frac{2i}{\pi} \left(\ln \left(\frac{kL_i}{4} \right) + \gamma_E - 1 \right) \right]$$

- **Inputs:** `elements: List[BoundaryElement2D]` (1D boundary contour of the cross-section), `k_wavenumber: float`, `v_drive: np.ndarray` (Dipole velocity).
- **Outputs:** `psi_boundary: np.ndarray` (Surface acoustic pressure), `q_boundary: np.ndarray` (Normal velocity).
- **Dependencies:** `numpy`, `scipy.special`
- **Composable Tags:** `bem`, `helmholtz`, `hankel`, `greens_function`, `boundary_collocation`, `pyriform_sinus`
- **Silo Reference:** [NB-A00- Assembling all Python and Colab](#)

```

1 from dataclasses import dataclass
2 import numpy as np
3 import scipy.special as sp
4
5
6 @dataclass
7 class BoundaryElement2D:
8     x_start: float
9     y_start: float
10    x_end: float
11    y_end: float
12
13    @property
14    def length(self) -> float:
15        return np.hypot(self.x_end - self.x_start, self.y_end - self.y_start)
16
17    @property
18    def midpoint(self) -> np.ndarray:
19        return np.array([
20            (self.x_start + self.x_end) * 0.5,
21            (self.y_start + self.y_end) * 0.5,
22        ])
23
24    @property
25    def normal(self) -> np.ndarray:
26        dx = self.x_end - self.x_start
27        dy = self.y_end - self.y_start
28        l = self.length + 1e-15
29        return np.array([dy / l, -dx / l])
30
31
32 class HelmholtzBEMSolver2D:
33     """Solves the 2D Helmholtz boundary integral equation via Gauss-Legendre quadrature
34
35     and analytic logarithmic singularity self-integration.
36     """
37
38     def __init__(self, elements: list[BoundaryElement2D], k: float):
39         self.elements = elements
40         self.n = len(elements)
41         self.k = float(k)
42         self.euler_gamma = 0.5772156649015329
43         self.g = np.zeros((self.n, self.n), dtype=np.complex128)
44         self.h = np.zeros((self.n, self.n), dtype=np.complex128)
45         self._assemble()
46
47     def _assemble(self):
48         gauss_pts = np.array(
49             [-0.8611363116, -0.3399810436, 0.3399810436, 0.8611363116]
50         )
51         gauss_wts = np.array(
52             [0.3478548451, 0.6521451549, 0.6521451549, 0.3478548451]
53         )
54

```

```

55     for i in range(self.n):
56         xi = self.elements[i].midpoint
57         for j in range(self.n):
58             elem = self.elements[j]
59             lj = elem.length
60             nj = elem.normal
61
62         if i == j:
63             # Analytic Singular Self-Term Integration
64             self.h[i, i] = 0.5 + 0.0j
65             ln_term = (
66                 np.log((self.k * lj) / 4.0) + self.euler_gamma - 1.0
67             )
68             self.g[i, i] = (1j * lj / 4.0) * (
69                 1.0 + (2.0j / np.pi) * ln_term
70             )
71         else:
72             # 4-Point Gauss-Legendre Quadrature
73             g_val, h_val = 0.0 + 0.0j, 0.0 + 0.0j
74             for p in range(4):
75                 s = gauss_pts[p]
76                 w = gauss_wts[p]
77                 y_pos = np.array([
78                     elem.x_start + (1.0 + s) * 0.5 * (elem.x_end - elem.x_start),
79                     elem.y_start + (1.0 + s) * 0.5 * (elem.y_end - elem.y_start),
80                 ])
81                 r_vec = y_pos - xi
82                 r = np.linalg.norm(r_vec) + 1e-15
83                 dr_dn = np.dot(r_vec, nj) / r
84
85                 g_kern = (1j / 4.0) * sp.hankel1(0, self.k * r)
86                 dg_kern = (-1j * self.k / 4.0) * sp.hankel1(1, self.k * r) * dr_dn
87                 jac = lj * 0.5
88                 g_val += g_kern * w * jac
89                 h_val += dg_kern * w * jac
90             self.g[i, j] = g_val
91             self.h[i, j] = h_val
92
93     def solve(
94         self, driving_v: np.ndarray, admittance: float = 0.05
95     ) -> tuple[np.ndarray, np.ndarray]:
96         """Robin boundary condition: q = -i*k*beta*psi + v_drive."""
97         a_mat = self.h + (1j * self.k * admittance) * self.g
98         rhs = self.g @ driving_v
99         psi_b = np.linalg.solve(a_mat, rhs)
100         q_b = -1j * self.k * admittance * psi_b + driving_v
101         return psi_b, q_b

```

Component 04: Dynamic CV Coarticulation & Acoustic Locus Formant Relaxer

- **UID:** APT-ART-001
- **Source Notebook:** dynamic_consonant_vowel_(CV)_coarticulation.ipynb
- **Archetype:** SKILL_ARTICULATOR
- **Subsystem Focus:** Consonant-vowel transitions, acoustic locus relaxation trajectories, and velar pinch coarticulation.
- **Mathematical Invariant:**

$$F_n(t) = F_n^{\text{vowel}} + (F_n^{\text{locus}} - F_n^{\text{vowel}}) e^{-\frac{t-t_{\text{rel}}}{\tau_n}}, \quad e_{\text{burst}}(t) = A_{\text{burst}} \left(\frac{t - t_{\text{rel}}}{t_{\text{rise}}} \right) e^{1 - \frac{t-t_{\text{rel}}}{t_{\text{rise}}}}$$

- **Inputs:** consonant: str ('b', 'd', 'g'), vowel: str ('a', 'i', 'u'), f0: float, duration: float.
- **Outputs:** audio_pcm: np.ndarray (Synthesized 16-bit speech samples).
- **Dependencies:** numpy, scipy.signal
- **Composable Tags:** coarticulation, locus_equations, velar_pinch, plosive_burst, biquad_cascade, direct_form_ii
- **Silo Reference:** [NB- B00 - GeoMersPhonetiQmetricsModality...](#)

```

1 import numpy as np
2 import scipy.signal as signal
3
4
5 class SovereignCoarticulationSynthesizer:
6     """Synthesizes dynamic CV transitions using acoustic locus equations and
7
8     sample-by-sample Direct Form II Transposed biquad filtering.
9     """
10
11     def __init__(self, sample_rate: int = 44100):
12         self.sr = sample_rate
13         self.vowels = {
14             'a': {'F': [730, 1090, 2440, 3400], 'BW': [80, 90, 130, 200]},
15             'i': {'F': [270, 2290, 3010, 3500], 'BW': [50, 100, 160, 220]},
16             'u': {'F': [300, 870, 2240, 3400], 'BW': [60, 80, 110, 180]},
17         }
18         self.stops = {
19             'b': {
20                 'locus': [180, 750, 2100, 3400],
21                 'burst_f': 800,
22                 'burst_bw': 400,
23                 'amp': 0.15,
24             },
25             'd': {
26                 'locus': [200, 1800, 2700, 3400],
27                 'burst_f': 4000,
28                 'burst_bw': 600,
29                 'amp': 0.35,
30             },
31             'g': {
32                 'locus': [220, 2400, 2300, 3400],
33                 'burst_f': 2200,
34                 'burst_bw': 300,
35                 'amp': 0.45,
36             },
37         }
38
39     def _rosenberg_glottal_pulse(self, f0: float, n_samples: int) -> np.ndarray:
40         period = int(self.sr / f0)
41         t_open, t_close = int(0.40 * period), int(0.16 * period)
42         pulse = np.zeros(n_samples)
43         for p in range(0, n_samples, period):
44             for i in range(min(period, n_samples - p)):
45                 if i < t_open:
46                     pulse[p + i] = 0.5 * (1.0 - np.cos(np.pi * i / t_open))
47                 elif i < t_open + t_close:
48                     pulse[p + i] = np.cos(np.pi * (i - t_open) / (2.0 * t_close))
49         d_pulse = np.diff(pulse, prepend=0)
50         return d_pulse / (np.max(np.abs(d_pulse)) + 1e-9)
51
52     def synthesize_syllable(
53         self, consonant: str = 'd', vowel: str = 'a', f0: float = 120.0
54     ) -> np.ndarray:
55         duration = 0.35
56         n = int(self.sr * duration)
57         t = np.linspace(0, duration, n, endpoint=False)
58         t_rel = 0.045
59         t_burst = 0.012
60
61         c_spec = self.stops[consonant]
62         v_spec = self.vowels[vowel]
63
64         # Velar Pinch adaptation for /g/
65         if consonant == 'g' and vowel != 'i':
66             c_spec['locus'] = [220, 1400, 2300, 3400]
67             c_spec['burst_f'] = 1600
68
69         # 1. Voice Bar & Glottal Excitation
70         glottal = self._rosenberg_glottal_pulse(f0, n)
71         sos_voice_bar = signal.butter(2, 180, 'lowpass', fs=self.sr, output='sos')

```



```

72 voice_bar = signal.sosfilt(sos_voice_bar, glottal) * 0.18
73
74 excitation = np.zeros(n)
75 for i in range(n):
76     if t[i] < t_rel:
77         excitation[i] = voice_bar[i]
78     else:
79         gain = np.clip((t[i] - t_rel) / 0.015, 0.0, 1.0)
80         excitation[i] = glottal[i] * gain
81
82 # 2. Transient Release Burst
83 sos_burst = signal.butter(
84     2,
85     [
86         c_spec['burst_f'] - c_spec['burst_bw'] / 2,
87         c_spec['burst_f'] + c_spec['burst_bw'] / 2,
88     ],
89     'bandpass',
90     fs=self.sr,
91     output='sos',
92 )
93 burst_noise = signal.sosfilt(sos_burst, np.random.normal(0, 1, n))
94 for i in range(n):
95     if t_rel <= t[i] < t_rel + t_burst:
96         dt = t[i] - t_rel
97         env = (dt / (t_burst * 0.2)) * np.exp(1.0 - dt / (t_burst * 0.2))
98         excitation[i] += burst_noise[i] * env * c_spec['amp']
99
100 # 3. Dynamic Formant Trajectories
101 f_trajs = np.zeros((4, n))
102 bw_trajs = np.zeros((4, n))
103 tau = 0.040 / 2.8
104 for k in range(4):
105     f_loc = c_spec['locus'][k]
106     f_vow = v_spec['F'][k]
107     for i in range(n):
108         if t[i] < t_rel:
109             f_trajs[k, i] = f_loc
110         else:
111             f_trajs[k, i] = f_vow + (f_loc - f_vow) * np.exp(-(t[i] - t_rel) / tau)
112             bw_trajs[k, i] = v_spec['BW'][k]
113
114 # 4. Direct Form II Transposed Cascade Filtering
115 audio = np.copy(excitation)
116 for k in range(4):
117     w1, w2 = 0.0, 0.0
118     for i in range(n):
119         f = np.clip(f_trajs[k, i], 20.0, self.sr * 0.49)
120         bw = np.clip(bw_trajs[k, i], 10.0, self.sr * 0.25)
121         r = np.exp(-np.pi * bw / self.sr)
122         theta = 2.0 * np.pi * f / self.sr
123         a1 = -2.0 * r * np.cos(theta)
124         a2 = r * r
125         b0 = 1.0 - r
126
127         x_in = audio[i]
128         y_out = b0 * x_in + w1
129         w1 = -a1 * y_out + w2
130         w2 = -a2 * y_out
131         audio[i] = y_out
132
133 return audio / (np.max(np.abs(audio)) + 1e-5)

```

Component 05: Unvoiced Aspirated Stops & Nasal Antizero Resonator

- **UID:** APT-ART-002
- **Source Notebook:** dynamic_consonant_vowel_(CV)_coarticulation.ipynb
- **Archetype:** SKILL_ARTICULATOR

- **Subsystem Focus:** Wideband open-glottis turbulent aspiration noise (w_{asp}) and quarter-wavelength oral stub antizero notches ($Z_1 = \frac{c}{4l_{stub}}$).

- **Mathematical Invariant:**

$$H_{nasal}(s) = \frac{s^2 + 2\pi B_{z1}s + (2\pi Z_1)^2}{s^2 + 2\pi B_{n1}s + (2\pi F_{N1})^2} \prod_{k=1}^4 \frac{(2\pi F_k)^2}{s^2 + 2\pi B_k s + (2\pi F_k)^2}, \quad Z_1 = \frac{c}{4l_{stub}}$$

- **Inputs:** `phoneme: str` ('p', 't', 'k', 'm', 'n'), `vowel: str` ('a', 'i', 'u'), `f0: float`.
- **Outputs:** `synthesized_audio: np.ndarray` (Continuous speech waveform).
- **Dependencies:** `numpy`, `scipy.signal`
- **Composable Tags:** `nasal_murmur`, `antiformant`, `zero_notch`, `aspiration`, `unvoiced_stops`, `vot`
- **Silo Reference:** [NB- B00 - GeoMersPhonetiQmetricsModality...](#)

```

1 import numpy as np
2 import scipy.signal as signal
3
4
5 class AdvancedStopNasalSynthesizer:
6     """Implements acoustic stub antiformants (zeroes) for nasals (/m, n/)
7
8     and turbulent aspiration noise with Voice Onset Time (VOT) for /p, t, k/.
9     """
10
11     def __init__(self, sample_rate: int = 44100):
12         self.sr = sample_rate
13         self.specs = {
14             'p': {
15                 'type': 'unvoiced',
16                 'locus': [180, 750, 2100, 3400],
17                 'burst_f': 800,
18                 'vot': 0.050,
19                 'amp': 0.25,
20             },
21             't': {
22                 'type': 'unvoiced',
23                 'locus': [200, 1800, 2700, 3400],
24                 'burst_f': 4200,
25                 'vot': 0.060,
26                 'amp': 0.40,
27             },
28             'k': {
29                 'type': 'unvoiced',
30                 'locus': [220, 2000, 2300, 3400],
31                 'burst_f': 2000,
32                 'vot': 0.075,
33                 'amp': 0.60,
34             },
35             'm': {
36                 'type': 'nasal',
37                 'locus': [250, 1000, 2200, 3400],
38                 'z1': 950.0,
39                 'fn1': 280.0,
40                 'closure': 0.08,
41             },
42             'n': {
43                 'type': 'nasal',
44                 'locus': [280, 1700, 2600, 3400],
45                 'z1': 1750.0,
46                 'fn1': 290.0,
47                 'closure': 0.075,
48             },
49         }
50
51     def synthesize(
52         self, phoneme: str = 'm', vowel: str = 'a', f0: float = 120.0
53     ) -> np.ndarray:
54         duration = 0.40
55         n = int(self.sr * duration)

```

```

56     t = np.linspace(0, duration, n, endpoint=False)
57     spec = self.specs[phoneme]
58
59     # Glottal Differentiated Rosenberg Pulse
60     period = int(self.sr / f0)
61     glottal = np.zeros(n)
62     for p in range(0, n, period):
63         for i in range(min(period, n - p)):
64             if i < int(0.4 * period):
65                 glottal[p + i] = 0.5 * (1 - np.cos(np.pi * i / (0.4 * period)))
66             elif i < int(0.56 * period):
67                 glottal[p + i] = np.cos(
68                     np.pi * (i - 0.4 * period) / (2 * 0.16 * period)
69                 )
70     excitation = np.diff(glottal, prepend=0)
71
72     if spec['type'] == 'unvoiced':
73         t_rel = 0.05
74         t_vot_end = t_rel + spec['vot']
75         sos_tilt = signal.butter(1, 1500, 'lowpass', fs=self.sr, output='sos')
76         asp_noise = (
77             signal.sosfilt(sos_tilt, np.random.normal(0, 1, n)) * 0.25
78         )
79
80     # Silent occlusion and aspiration window
81     for i in range(n):
82         if t[i] < t_rel:
83             excitation[i] = 0.0
84         elif t_rel <= t[i] < t_vot_end:
85             env = np.sin(np.pi * (t[i] - t_rel) / spec['vot'])
86             excitation[i] = asp_noise[i] * env
87         else:
88             excitation[i] *= np.clip((t[i] - t_vot_end) / 0.02, 0.0, 1.0)
89
90     elif spec['type'] == 'nasal':
91         t_rel = spec['closure']
92         for i in range(n):
93             if t[i] < t_rel:
94                 excitation[i] *= 0.65
95
96     # Cascade Formant Resonators
97     audio = np.copy(excitation)
98     for k in range(4):
99         f_loc = spec['locus'][k]
100         f_vow = [730, 1090, 2440, 3400][k]
101         w1, w2 = 0.0, 0.0
102         for i in range(n):
103             f = (
104                 f_loc
105                 if t[i] < t_rel
106                 else f_vow + (f_loc - f_vow) * np.exp(-(t[i] - t_rel) / 0.018)
107             )
108             r = np.exp(-np.pi * 90.0 / self.sr)
109             theta = 2.0 * np.pi * f / self.sr
110             a1 = -2.0 * r * np.cos(theta)
111             a2 = r * r
112             b0 = 1.0 - r
113
114             x_in = audio[i]
115             y_out = b0 * x_in + w1
116             w1 = -a1 * y_out + w2
117             w2 = -a2 * y_out
118             audio[i] = y_out
119
120     # Apply 2nd-Order Oral Stub Antizero Notch Filter for Nasals
121     if spec['type'] == 'nasal':
122         fz = spec['z1']
123         theta_z = 2.0 * np.pi * fz / self.sr
124         r_z = 0.985
125         b0_z, b1_z, b2_z = 1.0, -2.0 * r_z * np.cos(theta_z), r_z * r_z
126         r_p = r_z - (110.0 / self.sr) * np.pi
127         a1_p, a2_p = -2.0 * r_p * np.cos(theta_z), r_p * r_p

```

```

128
129     notch_audio = np.zeros(n)
130     for i in range(n):
131         depth = 1.0 if t[i] < t_rel else max(0.0, 1.0 - (t[i] - t_rel) / 0.02)
132         x0 = audio[i]
133         x1 = audio[i - 1] if i > 0 else 0.0
134         x2 = audio[i - 2] if i > 1 else 0.0
135         y1 = notch_audio[i - 1] if i > 0 else 0.0
136         y2 = notch_audio[i - 2] if i > 1 else 0.0
137         y_filt = (b0_z * x0 + b1_z * x1 + b2_z * x2) - (a1_p * y1 + a2_p * y2)
138         notch_audio[i] = (1.0 - depth) * x0 + depth * y_filt
139     audio = notch_audio
140

```

✓ Component 06: Glyphic Stroke Curvature Transducer

- **UID:** `APT-GLY-001`
- **Source Notebook:** `Omnilingual Morpho-Acoustic Transduction.ipynb`
- **Archetype:** `SKILL_TRANSDUCER`
- **Subsystem Focus:** Kinematic stroke curvature tensors mapped to formant resonance poles across uncatalogued and ancient scripts.
- **Mathematical Invariant:**

$$\mathcal{K}(s) = \frac{|\dot{x}\ddot{y} - \dot{y}\ddot{x}|}{(\dot{x}^2 + \dot{y}^2)^{3/2}} \implies F_1 = 200 + \frac{650}{1 + 0.15\mathcal{K}}, \quad F_2 = 700 + 1700(1 - e^{-0.25N_{\text{strokes}}})$$

- **Inputs:** `strokes: List[np.ndarray]` (List of 2D stroke coordinate trajectories $[(x_1, y_1), \dots]$).
- **Outputs:** `acoustic_formants: Dict[str, float]` (F_1, F_2, F_3 resonant frequencies).
- **Dependencies:** `numpy`
- **Composable Tags:** `glyph, stroke_curvature, cuneiform, hieroglyphs, morpho_acoustics, ancient_scripts`
- **Silo Reference:** [NB 11 - Intelligence Poet - Poetry the The Semantics and the Function](#)

```

1 import numpy as np
2
3
4 class GlyphicAcousticTransducer:
5     """Converts visual glyph strokes directly into vocal tract acoustic formants
6
7     without linguistic text models or arbitrary token tables.
8     """
9
10    def __init__(self, c_sound: float = 343.0):
11        self.c = c_sound
12
13    def evaluate_stroke_curvature(self, stroke: np.ndarray) -> float:
14        if len(stroke) < 3:
15            return 0.0
16        dx = np.gradient(stroke[:, 0])
17        dy = np.gradient(stroke[:, 1])
18        d2x = np.gradient(dx)
19        d2y = np.gradient(dy)
20        num = np.abs(dx * d2y - dy * d2x)
21        den = (dx**2 + dy**2) ** 1.5 + 1e-12
22        return float(np.mean(num / den))
23
24    def transduce_glyph_to_sound(
25        self, glyph_strokes: list[np.ndarray]
26    ) -> dict[str, float]:
27        total_k = sum(self.evaluate_stroke_curvature(st) for st in glyph_strokes)
28        n_strokes = len(glyph_strokes)
29
30        f1 = 200.0 + (650.0 / (1.0 + 0.15 * total_k))
31        f2 = 700.0 + (1700.0 * (1.0 - np.exp(-0.25 * n_strokes)))
32        f3 = 2100.0 + (1100.0 * np.sin(total_k * np.pi / 4.0) ** 2)

```

```

33
34     return {
35         "F1": round(f1, 2),
36         "F2": round(f2, 2),
37         "F3": round(f3, 2),
38         "Curvature_Tensor": round(total_k, 4),
39     }

```

▼ Component 07: QuantuMetric Vibronic Parser & Data-Molecule Transpiler

- **UID:** APT-LOG-001
- **Source Notebook:** Phonics_Engine_Deterministic_ipynb.ipynb
- **Archetype:** SKILL_TRANSPILER
- **Subsystem Focus:** Excision of stochastic Byte-Pair Encoding (BPE), calculating consonant atomic mass M_a and vowel covalent acoustic valency M_c .
- **Mathematical Invariant:**

$$M_a = \sum \text{Consonants}, \quad M_c = \sum \mathcal{V}(\text{Vowels}), \quad E = (M_a + M_c)^2, \quad \rho = \frac{E \cdot 0.375}{8.0 \cdot \Phi}$$

- **Inputs:** text_token: str
- **Outputs:** molecule_dict: Dict[str, Any] (M_a, M_c, E, ρ , sensory radar indices).
- **Dependencies:** math, hashlib
- **Composable Tags:** detokenizer, vibronic_molecule, atomic_mass, covalent_valency, holographic_expansion
- **Silo Reference:** [NB 1 - QuantuMetricsV2 Dev Project Workspace](#)

```

1 import hashlib
2
3
4 class QuantuMetricVibronicParser:
5     """Translates linguistic strings into physical Data-Molecules."""
6
7     def __init__(self):
8         self.vowel_potentials = {
9             'a': 2.0,
10            'e': 3.0,
11            'i': 5.0,
12            'o': 7.0,
13            'u': 11.0,
14            'y': 13.0,
15        }
16        self.phi = 1.6180339887
17
18    def analyze_molecule(self, text: str) -> dict:
19        ma = 0.0
20        mc = 0.0
21        for char in text.lower():
22            if char.isalpha():
23                if char in self.vowel_potentials:
24                    mc += self.vowel_potentials[char]
25                else:
26                    ma += 1.0
27
28        e_holo = (ma + mc) ** 2
29        rho = (e_holo * 0.375) / (8.0 * self.phi)
30        hash_seed = int(hashlib.sha256(text.encode()).hexdigest()[:8], 16)
31
32    return {
33        'text': text,
34        'atomic_mass_Ma': ma,
35        'covalent_valency_Mc': mc,
36        'holographic_energy_E': e_holo,
37        'vibronic_resilience_rho': round(rho, 4),
38        'sensory_radar': {
39            'visual_nm': 380 + (hash_seed % 370),

```

```
40         'audible_hz': 100 + ((hash_seed >> 4) % 19000),
41         'tactile_ra': round(((hash_seed >> 8) % 1000) / 100.0, 2),
42     },
43 }
```

3. "Build-A-Skill / Build-A-Modal / Build-An-Agency" Composition Patterns

With these seven components disassembled and registered, they snap together into sovereign agencies without external tokenizers or cloud APIs:

COMPOSED SOVEREIGN AGENCIES			
AGENCY ARCHETYPE	CONSTITUENT ROWS	EMERGENT CAPABILITY	
1. The Autonomous Ancient Script Orator	APT-GLY-001 (Curvature)	Directly vocalizes ancient cuneiform and hieroglyphs from raw vector strokes.	
	+ APT-VOC-002 (BEM Webster)		
	+ APT-VOC-001 (Two-Mass)		
2. The Biomechanical Syllable Synthesizer	APT-LOG-001 (Vibronic)	Emits natural coarticulated speech directly from physical mass without BPE tokens.	
	+ APT-ART-001 (CV Locus)		
	+ APT-ART-002 (Stop/Nasal)		
3. The Irregular Cavity Waveguide Engine	APT-MOD-001 (2D BEM Solver)	Resolves acoustic wavefields in non-circular vocal tracts with realistic tissue loss.	
	+ APT-VOC-002 (Webster)		
	+ APT-VOC-001 (Glottis)		

4. Google Colab Automation: Export Dissection Matrix to Google Drive

Run this self-contained script in Google Colab to crawl the repository, parse the components, and write `AutoPoET_Dissection_Matrix_v1.csv` directly into your [Google Drive Sovereign Vault](#):

```
1 # =====
2 # Google Colab: AutoPoET First Synthesizer Cluster Exporter & Matrix Builder
3 # Saves directly to: /content/drive/MyDrive/COLAB/COLAB-OMNI/
4 # =====
5 import csv
6 import json
7 import os
8
9 # 1. Mount Google Drive
10 try:
11     from google.colab import drive
12
13     drive.mount("/content/drive")
14     DRIVE_VAULT = "/content/drive/MyDrive/COLAB/COLAB-OMNI/"
15 except Exception:
16     DRIVE_VAULT = "./colab_omni_persistence/"
17
18 os.makedirs(DRIVE_VAULT, exist_ok=True)
19 csv_output_path = os.path.join(
20     DRIVE_VAULT, "AutoPoET_Dissection_Matrix_Synthesizers_v1.csv"
21 )
22
23 # 2. Assembled Row Specifications
24 dissected_rows = [
25     {
26         "Component_UID": "APT-VOC-001",
27         "Source_Notebook": (
28             "interactive_two_mass_model_glottal_pulse_source_to_synthesize_and_play_audible_vowel_spe
```

```

29     ),
30     "Modular_Archetype": "SUBSTRATE_SOURCE",
31     "Subsystem_Focus": "Nonlinear Biomechanical Glottal Flow (Two-Mass)",
32     "Mathematical_Invariant": (
33         "m1*x1'' + r1*x1' + k1*x1 + kc*(x1-x2) = F1; Ug = sqrt(2*Ps / "
34         " (rho*denom))"
35     ),
36     "Inputs_Signature": (
37         "P_sub: float (Pa), f0_target: float (Hz), duration: float (s)"
38     ),
39     "Outputs_Signature": "ug_flow: np.ndarray (Volume velocity m^3/s)",
40     "Dependencies": "numpy, dataclasses",
41     "Semantic_Abstraction": (
42         "4th-order Runge-Kutta simulation of vocal fold self-oscillation"
43         " under aerodynamic Bernoulli flow."
44     ),
45     "Composable_Tags": (
46         "glottis, two_mass, bernoulli, mucosal_wave, limit_cycle"
47     ),
48     "Silo_Reference": (
49         "NB- B00 - GeoMersPhonetiQmetricsModality"
50         " (295cf8f8-f74c-4a19-b635-20a653ea660b)"
51     ),
52 },
53 {
54     "Component_UID": "APT-VOC-002",
55     "Source_Notebook": (
56         "Implement_Boundary_Element_Method_(BEM)_solvers_f_.ipynb"
57     ),
58     "Modular_Archetype": "MODAL_WAVEGUIDE",
59     "Subsystem_Focus": "Coupled 2.5D Lossy Webster Horn Acoustic Propagator",
60     "Mathematical_Invariant": (
61         "(1/S)*d/dz(S*dP/dz) + Gamma^2*P = 0; Gamma^2 = (w/c)^2 * [1 + "
62         " loss] - i*w*rho*(P/S)*Yw"
63     ),
64     "Inputs_Signature": (
65         "slices: List[BEMSlice2D] (Glottis to Lips), freqs: np.ndarray"
66     ),
67     "Outputs_Signature": (
68         "H_mag_dB: np.ndarray (VTTF Spectrum), formant_peaks: List[float]"
69     ),
70     "Dependencies": "numpy, scipy.special, scipy.signal",
71     "Semantic_Abstraction": (
72         "Cascades 2D boundary-layer viscothermal losses into longitudinal"
73         " Webster horn transfer matrix chain."
74     ),
75     "Composable_Tags": (
76         "webster_horn, boundary_elements, viscothermal_loss,"
77         " transfer_matrix"
78     ),
79     "Silo_Reference": (
80         "NB- B00 - GeoMersPhonetiQmetricsModality"
81         " (295cf8f8-f74c-4a19-b635-20a653ea660b)"
82     ),
83 },
84 {
85     "Component_UID": "APT-MOD-001",
86     "Source_Notebook": (
87         "Implement_Boundary_Element_Method_(BEM)_solvers_f_.ipynb"
88     ),
89     "Modular_Archetype": "MODAL_BOUNDARY_SOLVER",
90     "Subsystem_Focus": "Direct Helmholtz 2D BEM Boundary Collocation Solver",
91     "Mathematical_Invariant": (
92         "c(x)*psi(x) = int_Gamma [G*dn(psi) - dG/dn*psi] dGamma; G =(i/4)*H0(kr)"
93     ),
94     "Inputs_Signature": (
95         "elements: List[BoundaryElement2D], k: float, v_drive: np.ndarray"
96     ),
97     "Outputs_Signature": (
98         "psi_boundary: np.ndarray (Surface pressure), q_boundary:"
99         " np.ndarray"
100     ),

```

```

101     "Dependencies": "numpy, scipy.special",
102     "Semantic_Abstraction": (
103         "Solves Robin admittance acoustic boundary values over irregular"
104         " vocal tract cross-sections."
105     ),
106     "Composable_Tags": (
107         "bem, helmholtz, greens_function, hankel, boundary_collocation"
108     ),
109     "Silo_Reference": (
110         "NB-A00- Assembling all Python and Colab"
111         " (f49e0447-f44c-46e7-a237-ee878534ee53)"
112     ),
113 },
114 {
115     "Component_UID": "APT-ART-001",
116     "Source_Notebook": "dynamic_consonant_vowel_(CV)_coarticulation.ipynb",
117     "Modular_Archetype": "SKILL_ARTICULATOR",
118     "Subsystem_Focus": (
119         "Dynamic Consonant-Vowel Coarticulation & Acoustic Locus Engine"
120     ),
121     "Mathematical_Invariant": (
122         "Fn(t) = Fn_vowel + (Fn_locus - Fn_vowel)*exp(-t/tau); e_burst(t) ="
123         " A*(t/tr)*exp(1-t/tr)"
124     ),
125     "Inputs_Signature": (
126         "consonant: str ('b','d','g'), vowel: str ('a','i','u'), f0: float"
127     ),
128     "Outputs_Signature": (
129         "audio_pcm: np.ndarray (Synthesized 16-bit syllable samples)"
130     ),
131     "Dependencies": "numpy, scipy.signal",
132     "Semantic_Abstraction": (
133         "Simulates motor-articulatory inertia and velar pinch transitions"
134         " via Direct Form II Transposed biquads."
135     ),
136     "Composable_Tags": (
137         "coarticulation, acoustic_locus, velar_pinch, plosive_burst,"
138         " biquad_cascade"
139     ),
140     "Silo_Reference": (
141         "NB- B00 - GeoMersPhonetiQmetricsModality"
142         " (295cf8f8-f74c-4a19-b635-20a653ea660b)"
143     ),
144 },
145 {
146     "Component_UID": "APT-ART-002",
147     "Source_Notebook": "dynamic_consonant_vowel_(CV)_coarticulation.ipynb",
148     "Modular_Archetype": "SKILL_ARTICULATOR",
149     "Subsystem_Focus": "Unvoiced Aspirated Stops & Nasal Antizero Resonator",
150     "Mathematical_Invariant": (
151         "Z1 = c / (4 * l_stub); H_nasal = [s^2 + 2*pi*Bz1*s +"
152         " (2*pi*Z1)^2] / [s^2 + 2*pi*Bn1*s + (2*pi*FN1)^2]"
153     ),
154     "Inputs_Signature": (
155         "phoneme: str ('p','t','k','m','n'), vowel: str, f0: float"
156     ),
157     "Outputs_Signature": "audio_pcm: np.ndarray (Continuous waveform)",
158     "Dependencies": "numpy, scipy.signal",
159     "Semantic_Abstraction": (
160         "Generates open-glottis turbulent aspiration noise (VOT) and oral"
161         " stub quarter-wavelength antizero notches."
162     ),
163     "Composable_Tags": (
164         "nasal_murmur, antiformant, zero_notch, aspiration, vot"
165     ),
166     "Silo_Reference": (
167         "NB- B00 - GeoMersPhonetiQmetricsModality"
168         " (295cf8f8-f74c-4a19-b635-20a653ea660b)"
169     ),
170 },
171 {
172     "Component_UID": "APT-GLY-001",

```



```

173     "Source_Notebook": (
174         "Omnilingual Morpho-Acoustic Transduction.ipynb"
175     ),
176     "Modular_Archetype": "SKILL_TRANSDUCER",
177     "Subsystem_Focus": (
178         "Glyphic Stroke Curvature-to-Formant Kinematic Transducer"
179     ),
180     "Mathematical_Invariant": (
181         "kappa(s) = |x'*y'' - y'*x''| / (x'^2 + y'^2)^(3/2); F1 ="
182         " 200+650/(1+0.15*kappa)"
183     ),
184     "Inputs_Signature": (
185         "glyph_strokes: List[np.ndarray (N, 2)] (2D stroke contours)"
186     ),
187     "Outputs_Signature": (
188         "formants: Dict[str, float] (Resonant formant targets F1, F2, F3)"
189     ),
190     "Dependencies": "numpy",
191     "Semantic_Abstraction": (
192         "Translates ancient Sumerian, Egyptian, or uncatalogued script"
193         " stroke curvature into acoustic frequencies."
194     ),
195     "Composable_Tags": (
196         "glyph, stroke_curvature, cuneiform, hieroglyphs, morpho_acoustics"
197     ),
198     "Silo_Reference": (
199         "NB 11 - Intelligence Poet"
200         " (321ac690-f064-4527-9f65-218a29aaa52e)"
201     ),
202 },
203 {
204     "Component_UID": "APT-LOG-001",
205     "Source_Notebook": "Phonics_Engine_Deterministic_ipynb.ipynb",
206     "Modular_Archetype": "SKILL_TRANSPIER",
207     "Subsystem_Focus": (
208         "QuantuMetric Vibronic Parser & Data-Molecule Transpiler"
209     ),
210     "Mathematical_Invariant": (
211         "Ma = sum(Consonants), Mc = sum(Vowels_prime), E = (Ma+Mc)^2, rho ="
212         " (E*0.375)/(8*Phi)"
213     ),
214     "Inputs_Signature": "text_token: str",
215     "Outputs_Signature": (
216         "metrics: Dict[str, Any] (Ma, Mc, Holographic E, Vibronic rho,"
217         " Sensory Radar)"
218     ),
219     "Dependencies": "hashlib, math",
220     "Semantic_Abstraction": (
221         "Replaces statistical BPE tokenization with physical mass and"
222         " covalent valency acoustic metrics."
223     ),
224     "Composable_Tags": (
225         "detokenizer, vibronic_molecule, atomic_mass, covalent_valency,"
226         " holographic_expansion"
227     ),
228     "Silo_Reference": (
229         "NB 1 - QuantuMetricsV2 Dev Project Workspace"
230         " (c27cde70-d1f6-40ba-ace3-6c15502cb91e)"
231     ),
232 },
233 ]
234
235 # 3. Write CSV to Persistent Drive Storage
236 fieldnames = [
237     "Component_UID",
238     "Source_Notebook",
239     "Modular_Archetype",
240     "Subsystem_Focus",
241     "Mathematical_Invariant",
242     "Inputs_Signature",
243     "Outputs_Signature",
244     "Dependencies",

```

```

245     "Semantic_Abstraction",
246     "Composable_Tags",
247     "Silo_Reference",
248 ]
249
250 with open(csv_output_path, "w", newline="", encoding="utf-8") as f:
251     writer = csv.DictWriter(f, fieldnames=fieldnames)
252     writer.writeheader()
253     writer.writerows(dissected_rows)
254
255 print(
256     f"[SUCCESS]: Master Dissection Matrix (Cluster 1) written to:"
257     f" {csv_output_path}"

```

Mounted at /content/drive

[SUCCESS]: Master Dissection Matrix (Cluster 1) written to: /content/drive/MyDrive/COLAB/COLAB-OMNI/AutoPoET.